Physics-Constrained Machine Learning

Victor M. Zavala and Angan Mukherjee

Department of Chemical and Biological Engineering
University of Wisconsin-Madison

victor.zavala@wisc.edu

Module 2: Physics-Informed ML Architectures

Concepts, Tools, and Algorithms

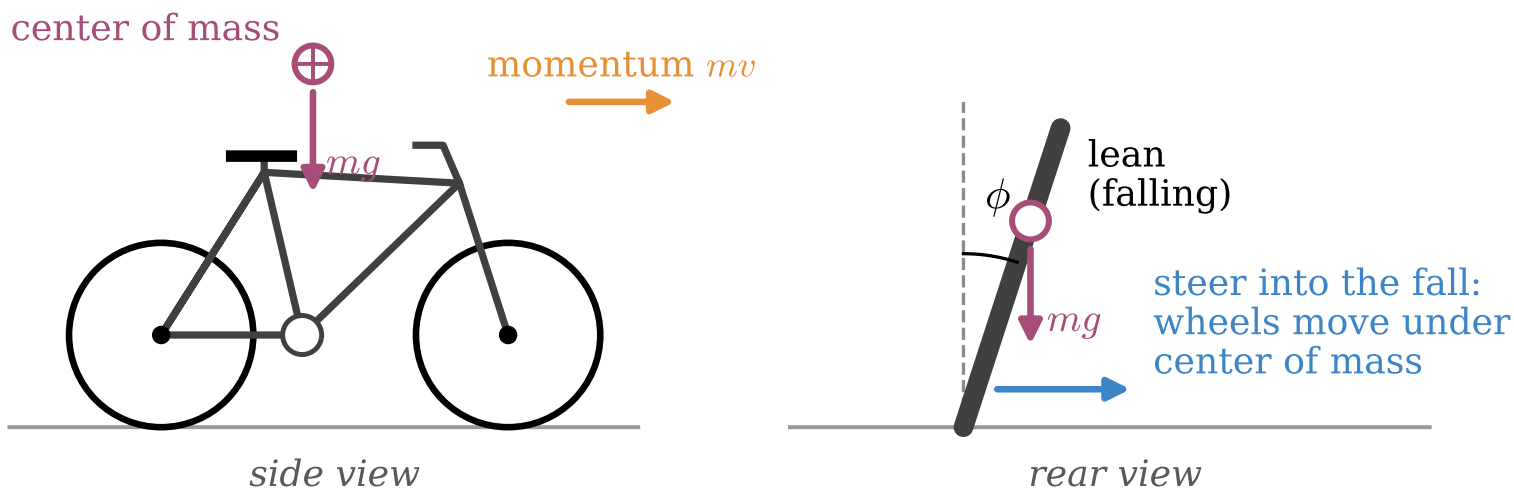


Why Physics?

- In Module 1 we saw that NNs and GPs are flexible and powerful tools that learn complex behavior directly from data.
- ML models are flexible, thus prone to *overfitting* and poor generalization (especially with limited data); in the CSTR, all models missed the temperature rise near ignition.
- In engineering we rarely start from zero; we often know (part of) the physics:
  - Conservation laws (mass and energy balances)
  - Thermodynamics and transport (equilibrium, property relations)
  - Kinetic mechanisms and their structure
  - Bounds and signs (e.g., concentrations are nonnegative)
- Physical knowledge, however, is typically incomplete or approximate.
- **Q:** How to leverage physical information in ML models and workflows?
- Physics *regularizes* ML models and *narrows down* the space in which we train; this improves reliability, generalization, interpretability, and data efficiency.

Analogy: Learning to Ride a Bicycle

- A kid that does not know physics learns to ride a bicycle by pure experimentation (data): many trials, many falls.
- The kid eventually succeeds, but cannot explain why steering into a fall or pedaling faster keeps the bicycle upright.
- Once the kid learns some physics (balance, momentum, center of mass), they can explain why they do what they do, and they need much less experimentation to master new situations (e.g., a different bicycle).
- ML models learn like the kid without physics: purely from data. Physics-constrained ML models learn like the kid who knows physics: less data needed, behavior easier to explain.



Ways to Incorporate Physics

- Physics can be incorporated at different stages of an ML workflow:
  - **Data:** use physics to select features and data representations.
  - **Model structure:** create hybrid models that combine physics and ML components (additive, composite, constrained).
  - **Training:** penalize violations of physics in the loss (physics-informed) or enforce physics as constraints (physics-constrained).
  - **Training space:** use physics to constrain, filter, or prioritize the domain where we collect data and train.
  - **Decision-making:** embed physics and ML models in optimization and experimental design (Module 4).
- These ideas apply to both NNs (including neural ODEs) and GPs.
- Approaches are not mutually exclusive; they are often combined.

Ways to Incorporate Physics: Examples

- We illustrate each approach with examples:

Approach	**Example**	**Physics used**
Features	Ionic liquids	Nernst-Einstein model and physical descriptors
Additive	Membrane separator	Approximate transport model + GP for its error
Physics-informed	Membrane separator	Mass balances penalized in loss
Physics-constrained	Membrane separator	Mass balances built into architecture
UDE (dynamic)	CSTR	Balance structure; tiny NN learns rate constant
Filtering	CSTR	Approximate model excludes unsafe region
How much physics?	Microbial communities	Consumer-resource model + NNs

- Notebooks: `PCML_Module2_Membrane` and `PCML_Module2_UDE_CSTR`.

Physics in the Data: Features and Representations

- Physics can be encoded in the data representation (ML model inputs).
- Examples: dimensionless groups, physical descriptors (e.g., density, viscosity), Arrhenius factors, molecular graphs, and sigma profiles.
- Physics-based representations facilitate learning and interpretability.
- CSTR: rate depends on $e^{-E/RT}$; this feature makes the input-output map nearly linear.

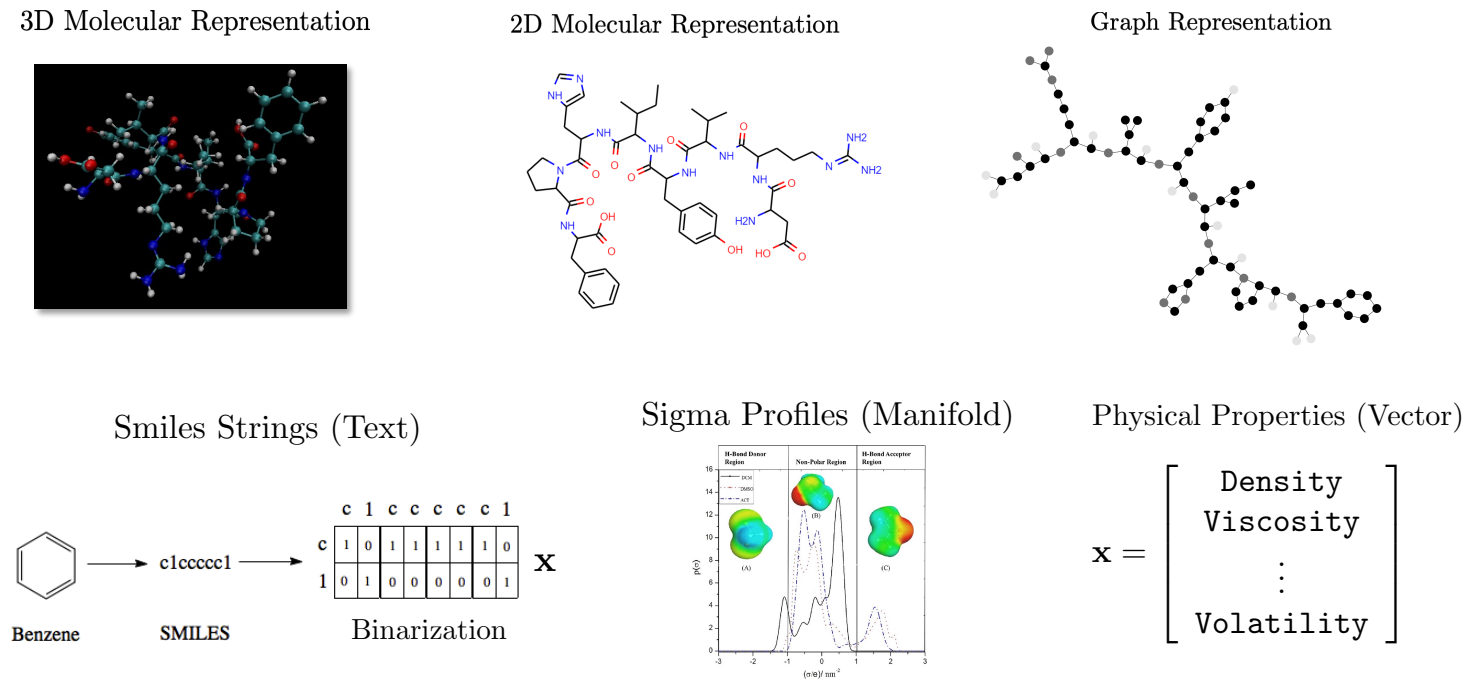


Figure: Different representations of molecules.

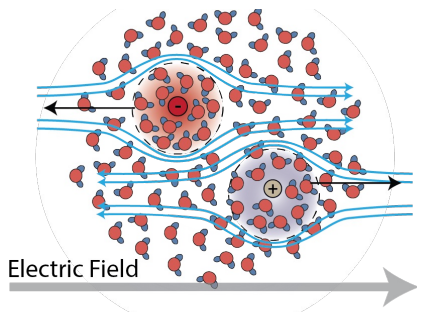
Example: Ionic Liquids for Batteries

- Ionic liquids (ILs) are promising candidates for safe battery electrolytes; huge molecular design space, but properties are not fully understood.
- Classical Nernst-Einstein (NE) model predicts ionic conductivity from viscosity μ and ion radii R_+, R_-:

$$\Lambda_{NE} = \frac{N_A e^2}{6\pi\mu}\left(\frac{1}{R_+} + \frac{1}{R_-}\right)$$

- NE model predicts conductivity of only certain ILs; many ILs deviate strongly from it.

Nerst-Einstein Model $\quad \Lambda_{NE} = \frac{N_A e^2}{6\pi\mu}\left(\frac{1}{R_+} + \frac{1}{R_-}\right)$



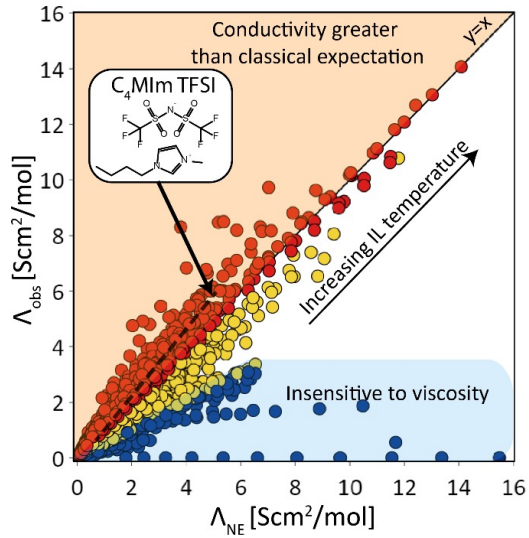
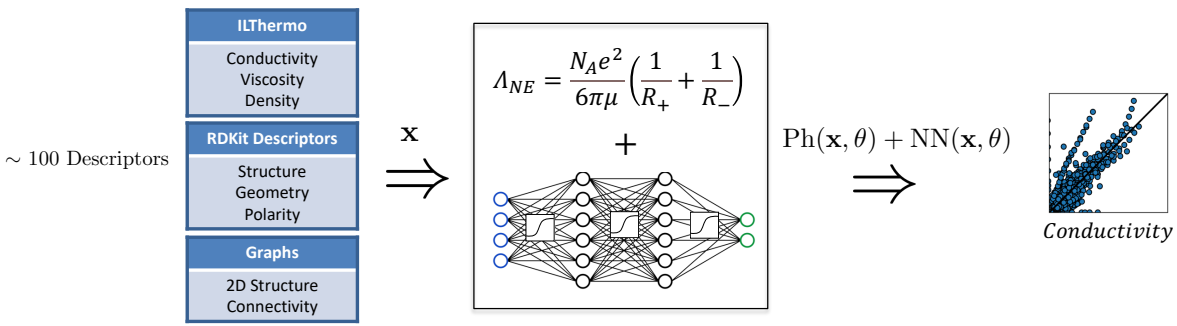


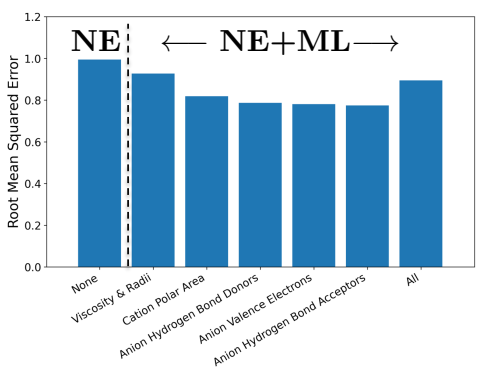
Figure: NE model (left) and observed vs. NE-predicted conductivity of ILs (right).

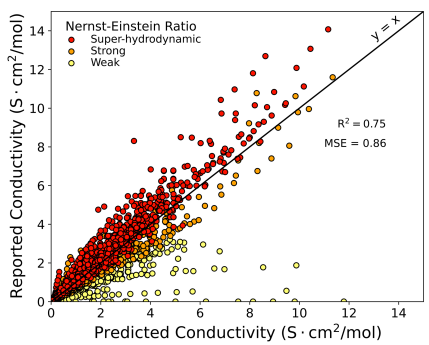
Umaña, J. E., Cashen, R. K., Zavala, V. M., and Gebbie, M. A. (2025). Uncovering ion transport mechanisms in ionic liquids using data science. Digital Discovery, 4(6), 1423-1436. doi:10.1039/d4dd00378k

Example: Physics-Informed Features for Ionic Liquids

- Approximately 100 descriptors: physical properties (conductivity, viscosity, density), molecular descriptors (structure, geometry, polarity), and graphs.
- Hybrid: NE model + NN with descriptors as features, $\mathrm{PH}(x) + \mathrm{ML}_\theta(x)$.
- Adding selected descriptors reduces error by about 20% relative to NE model; using all descriptors performs worse (feature selection matters).
- Molecular descriptors alone cannot fully explain conductivity; conductivity of some ILs results from collective effects.

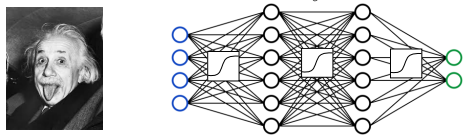






Hybrid Model Structures

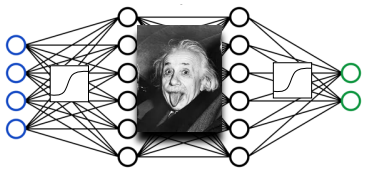
Additive Structure



$$y = \mathrm{PH}_\theta(x) + \mathrm{ML}_\theta(x)$$

- ML learns error of physics predictions

Composite Structure (Explicit)

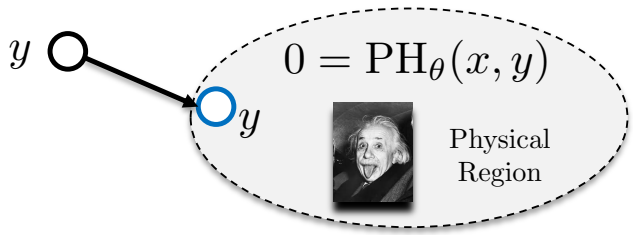


$$y = \mathrm{PH}_\theta(x, \mathrm{ML}_\theta(x))$$

- Enforces physics on ML predictions

Composite Structure (Implicit)

$$y = \mathrm{ML}_\theta(x) \qquad \text{(e.g., input-output flows)}$$
$$0 = \mathrm{PH}_\theta(x, y) \qquad \text{(e.g., mass balances)}$$



- Projects ML predictions to physical region

Mukherjee, A., and Zavala, V. M. (2026). Physics-constrained machine learning for chemical engineering. Curr. Opin. Chem. Eng., 51, 101228. doi:10.1016/j.coche.2026.101228

Example: Membrane Separator PCML_Module2_Membrane

- Membrane separates a liquid feed into permeate and retentate.
- Inputs: pressure $P \in [5, 15]$ bar, feed flow $F \in [50, 150]$ kg/h, and feed concentration $c_f \in [0.01, 0.03]$.
- Outputs: total flows F_p, F_r, solute flows S_p, S_r (kg/h); purities $c_p = S_p/F_p$, $c_r = S_r/F_r$.
- **Physics (exact):** no reaction, accumulation, or leaks; thus, for every input:

$$F_p + F_r = F, \qquad S_p + S_r = F\, c_f$$

- Balances do *not* tell how feed is split (depends on unknown transport).
- **Limited data:** only 10 noisy experiments for 3 inputs and 4 outputs (data generated with a solution-diffusion model that ML models do not know).

Example: Membrane Inputs, Outputs, and Parameters

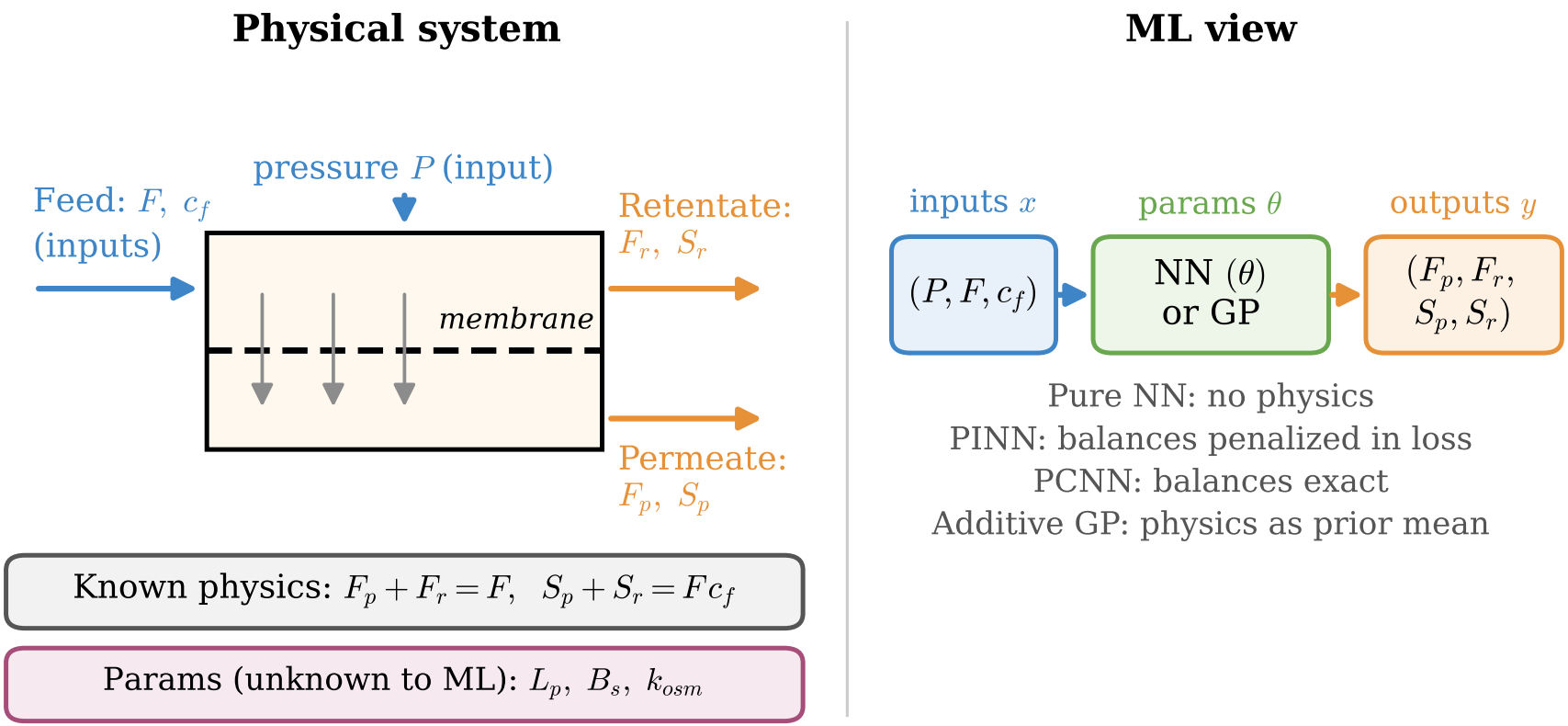


Figure: Membrane separator as a physical system (left) and as seen by ML models (right).

- Inputs $x = (P, F, c_f)$; outputs $y = (F_p, F_r, S_p, S_r)$; transport params (L_p, B_s, k_{osm}) are unknown to the ML models.
- Mass balances are known exactly; the hybrid models differ in how they use them.

Additive Hybrids

- Model is the sum of physics and an ML component (ML parts of hybrids in blue):

$$y = \mathrm{PH}(x) + \mathrm{ML}_\theta(x)$$

- **Learning:** ML learns the *error* of the physics model (residual $y - \mathrm{PH}(x)$); training algorithm does not change (Adam for NNs, MLE for GPs).

- **Requirements:** physics only needs to be evaluated at data points; it can be a black-box simulator (no derivatives needed).

- For GPs, additive structure is equivalent to using physics as the *prior mean*; far from data, GP reverts to physics instead of a constant.

- Outside data, predictions fall back on physics; extrapolation is as good as the physics.

- Physics captures coarse trends, so ML only needs to learn a simpler residual; simplest and most widely used hybrid structure.

Example: Additive GP for Membrane PCML_Module2_Membrane

- Approximate transport model (water permeance 15% too low) is used as prior mean of GP; GP learns its error.
- With 10 experiments, RMSE of F_p (kg/h): physics 4.6, pure GP 4.1, additive GP 1.5 (best).
- Pure GP has wide bands where data are scarce; additive GP follows physics.

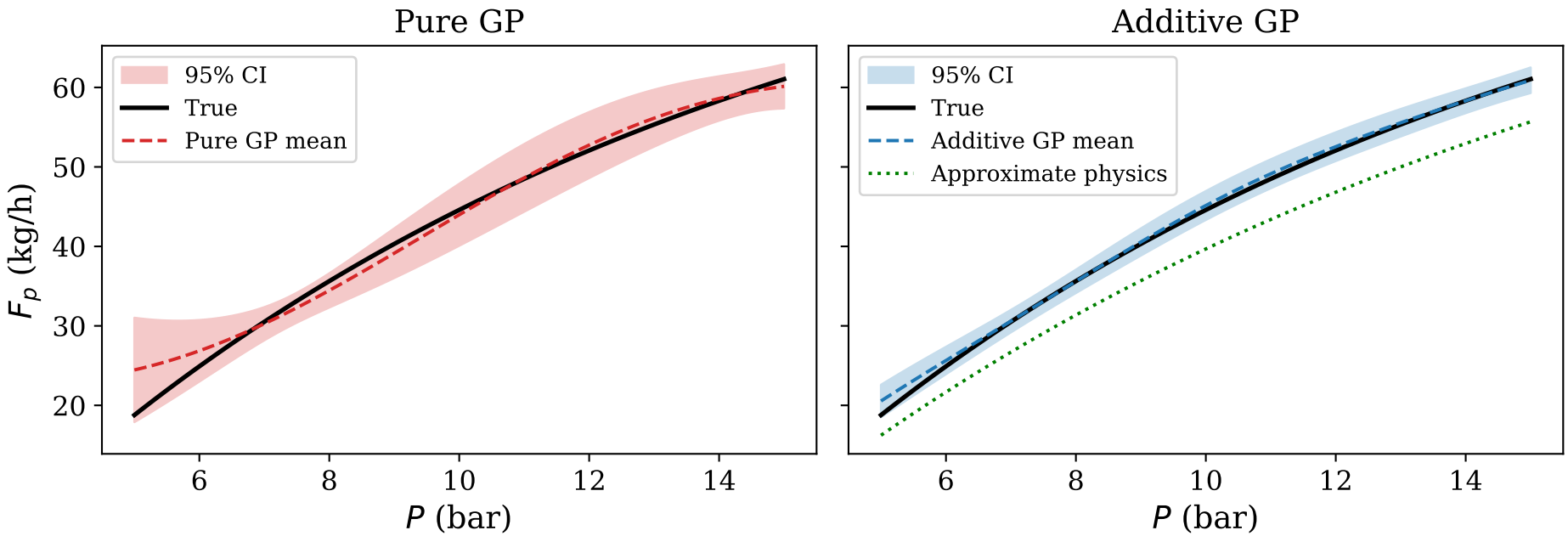


Figure: Permeate flow vs. pressure ($F = 100$ kg/h, $c_f = 0.02$) for pure GP (left) and additive GP (right) with 95% confidence intervals.

Physics-Informed NNs (PINNs)

- Keep ML architecture; add a *physics loss* penalizing violation of physics $\mathrm{PH}(x, y) = 0$:

$$\min_{\theta} \; L_d(\theta) + \lambda_p \, L_p(\theta)$$

with data and physics losses over the S observations (x_ω, y_ω), $\omega \in \mathcal{S}$:

$$L_d(\theta) = \frac{1}{2} \sum_{\omega \in \mathcal{S}} \left(y_\omega - \mathrm{ML}_\theta(x_\omega)\right)^2, \qquad L_p(\theta) = \frac{1}{2} \sum_{\omega \in \mathcal{S}} \mathrm{PH}(x_\omega, \mathrm{ML}_\theta(x_\omega))^2$$

- **Learning:** physics is a *regularizer* restricting functions the NN can represent; physics PH is enforced at all observations.
- **Requirements:** physics must be differentiable (PyTorch/JAX); for differential equations, NN output derivatives come from automatic differentiation.
- Optimization remains *unconstrained*; training algorithm does not change (Adam).
- Physics is satisfied only *approximately*; results depend on weight λ_p, and imbalance between loss terms can make training difficult.

Raissi, M., Perdikaris, P., and Karniadakis, G. E. (2019). Physics-informed neural networks. J. Comput. Phys., 378, 686-707. doi:10.1016/j.jcp.2018.10.045

Example: PINN for Membrane `PCML_Module2_Membrane`

- PINN penalizes violations of both mass balances at the 10 observations.
- Maximum violation of total balance on test inputs: pure NN 30 kg/h, PINN 24 kg/h.
- RMSE of F_p from 5.0 to 3.8 kg/h; RMSE of S_r from 0.19 to 0.14 kg/h.
- Physics holds only where it is enforced; away from the data, PINN can violate the balances.

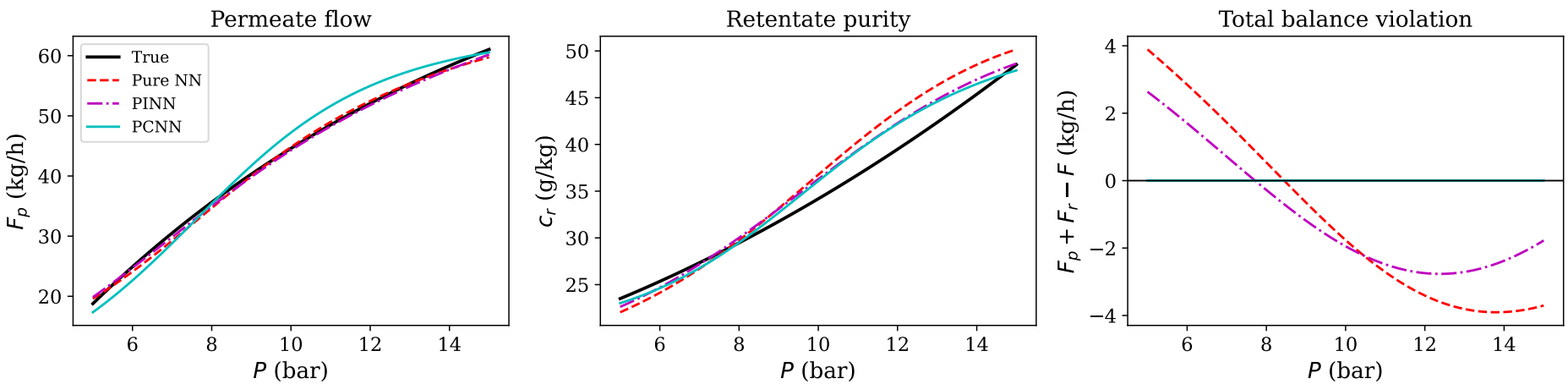


Figure: Permeate flow, retentate purity, and total balance violation vs. pressure ($F = 100$ kg/h, $c_f = 0.02$).

Physics-Constrained NNs (PCNNs)

- Enforce physics *exactly* (hard constraints):

$$\min_{\theta} \; L_d(\theta) \quad \text{s.t.} \quad \mathrm{PH}(x, y) = 0$$

- **Composite (explicit):** $y = \mathrm{PH}(x, \mathrm{ML}_\theta(x))$; ML predicts some quantities and physics computes the rest (e.g., NN predicts reaction rate inside mass balances, as in universal differential equations). Constraints hold for every input.

- **Composite (implicit):** $y = \mathrm{ML}_\theta(x)$, $0 = \mathrm{PH}(x, y)$; ML outputs projected onto physics.

- **Requirements:** explicit structures need differentiable physics; implicit ones need projection layers (sequential) or solvers like Ipopt (simultaneous).

- Physics reduces what the NN must learn (regularization); no penalty weight needed, but training is more complex and scalability is a challenge for large data and deep models.

Mukherjee, A., and Bhattacharyya, D. (2024). On the development of steady-state and dynamic mass-constrained neural networks using noisy transient data. Comput. Chem. Eng., 187, 108722. doi:10.1016/j.compchemeng.2024.108722

Example: PCNN for Membrane PCML_Module2_Membrane

- NN predicts recovery $R = F_p/F$ and split $s = S_p/(F c_f)$; balances give retentate flows:

$$\hat{F}_p = R\,F, \quad \hat{S}_p = s\,F\,c_f, \quad \hat{F}_r = F - \hat{F}_p, \quad \hat{S}_r = F\,c_f - \hat{S}_p$$

- Balances hold exactly for any input; NN learns two dimensionless outputs, not four flows.

Model	**RMSE** F_p (kg/h)	**RMSE** S_r (kg/h)	**RMSE** c_r (g/kg)	**Max violation** (kg/h)
Pure NN	5.0	0.19	5.3	30
PINN	3.8	0.14	3.3	24
PCNN	2.9	0.02	2.3	0

- Satisfying physics does not guarantee accuracy (split is still learned from data), but it regularizes the model and makes predictions physically consistent.

Example: How Much Data? Learning Curves `PCML_Module2_Membrane`

- Repeat study for $N = 5, 10, 20, 40$ experiments (average over 5 random designs).
- 10 experiments: purity RMSE 20.8 g/kg (pure NN), 3.6 (PINN), 2.8 (PCNN).
- Approximate physics (additive GP) gives accurate flows even with 5 experiments.
- As data accumulate, gap shrinks: physics matters most when data are scarce.

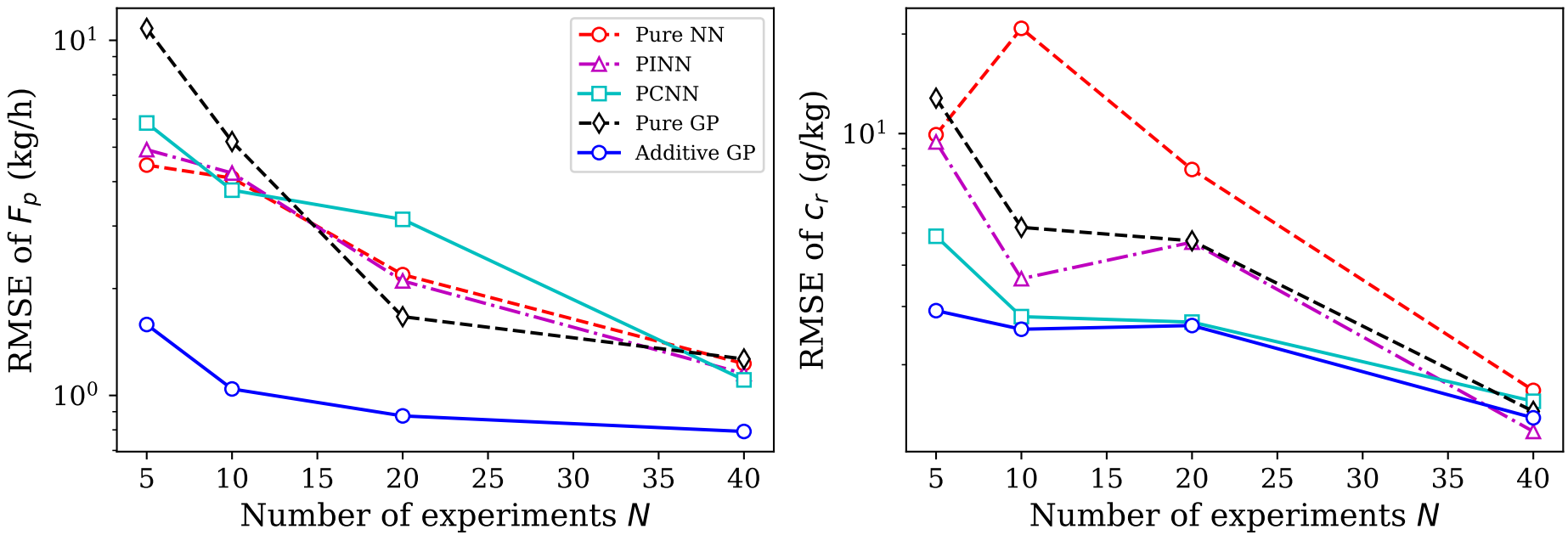


Figure: Average RMSE of permeate flow (left) and retentate purity (right) vs. number of experiments.

Universal Differential Equations (UDEs)

- Composite structure for dynamic systems: keep known terms of the ODEs and use NN only for unknown terms.
- CSTR: flow, reaction heat, and cooling terms known; rate $r = k(T)C_A$, unknown $k(T)$:

$$\frac{dC_A}{dt} = \frac{q}{V}(C_{Af} - C_A) - r$$
$$\frac{dT}{dt} = \frac{q}{V}(T_f - T) + \frac{(-\Delta H)}{\rho C_p} r + \frac{UA}{V\rho C_p}(T_c - T), \quad r = C_A\, k_{ref}\, e^{\mathrm{ML}_\theta(1/T)}$$

- System input is still T_c; inside the model, the NN takes $1/T$ (a physics-based feature of state T), has 2 neurons and one output: 7 params (vs. 98 for neural ODE of Module 1).
- **Learning:** trained exactly like a neural ODE (repeated simulation and fitting of trajectories); physics must be differentiable.
- NSM model for microbial communities is a UDE (see later).

Recall: CSTR Example PCML_Module2_UDE_CSTR

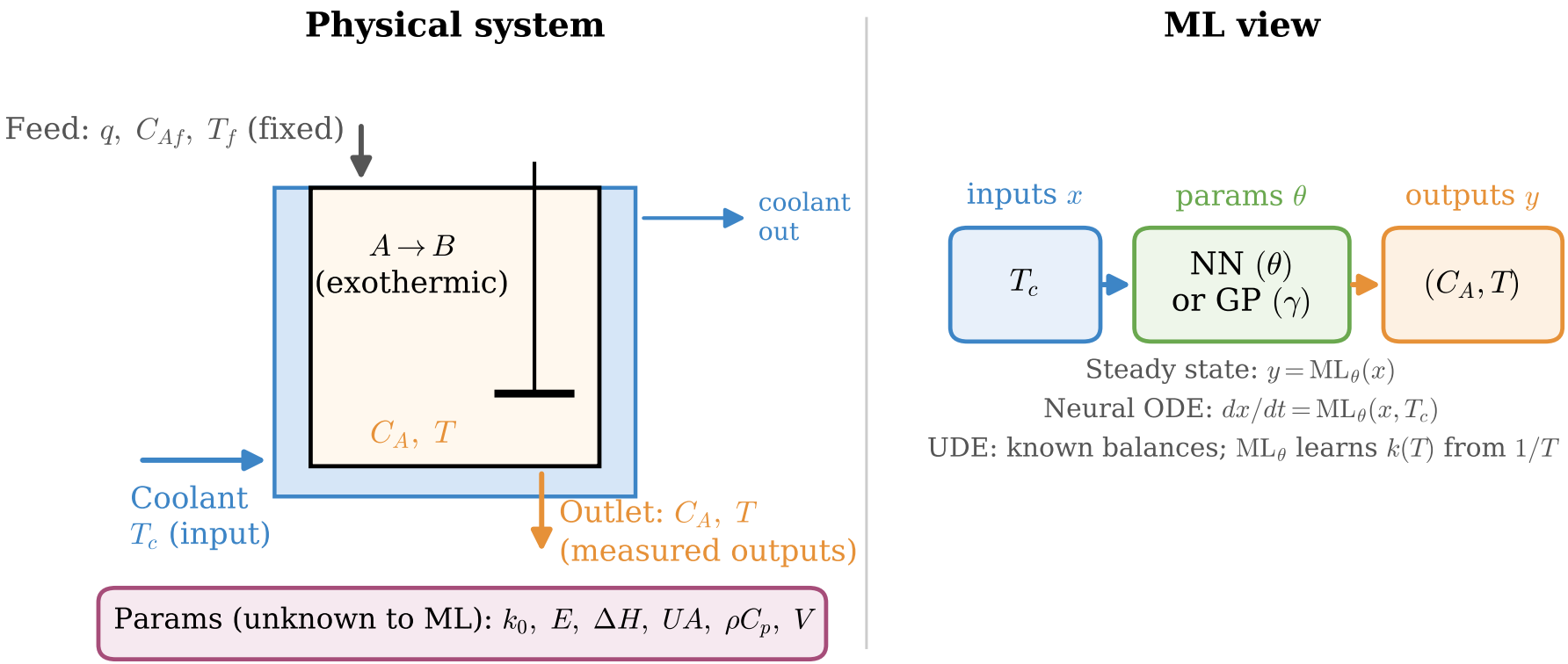


Figure: CSTR as a physical system (left) and as seen by ML models (right).

- Neural ODE learns the full dynamics of (C_A, T) from data (Module 1).
- UDE keeps known flow, heat, cooling terms; NN learns only $k(T)$ from $1/T$.

Example: UDE vs. Neural ODE for CSTR PCML_Module2_UDE_CSTR

- Both models trained with *a single experiment* (T_c stepped to 300 K); we predict response to five other values of T_c.
- Average RMSE of T: neural ODE 7.6 K (1 experiment), neural ODE 0.5 K (3 experiments), UDE 0.2 K (1 experiment).
- Flexible neural ODE overfits single experiment; physics regularizes UDE, reduces NN complexity (7 vs. 98 params), and mitigates limited data.

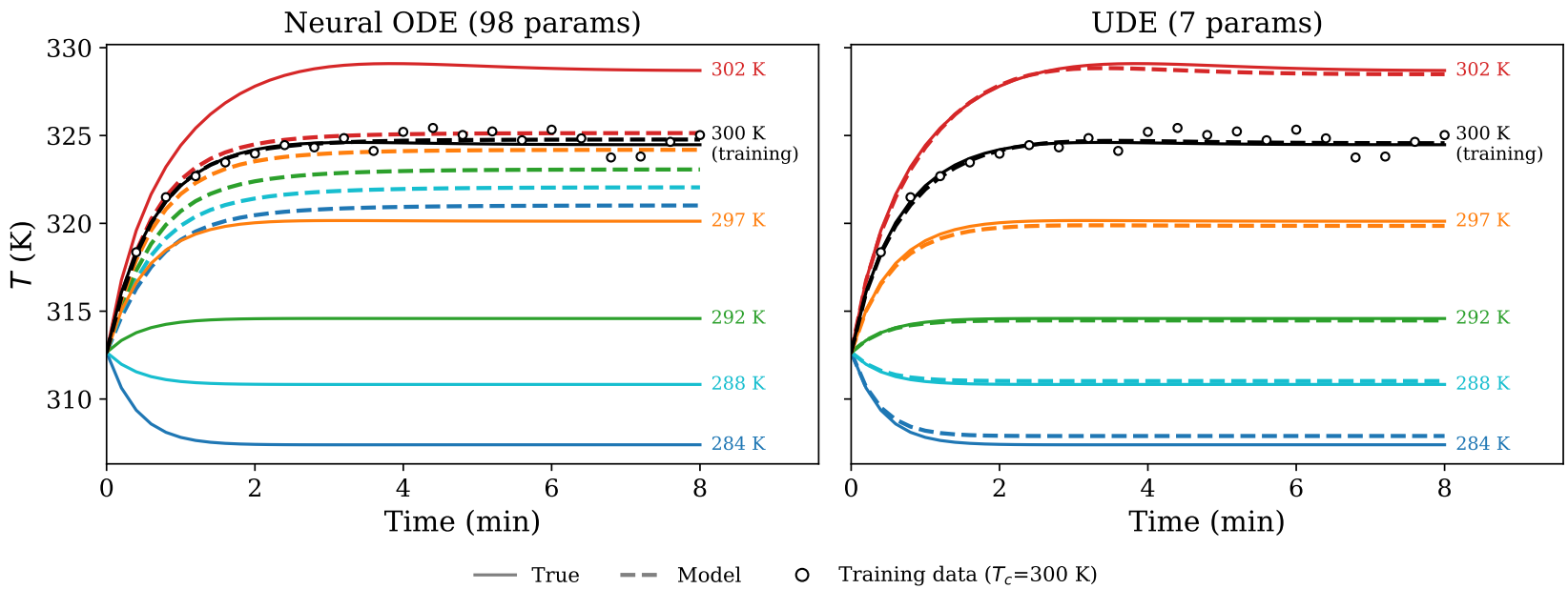


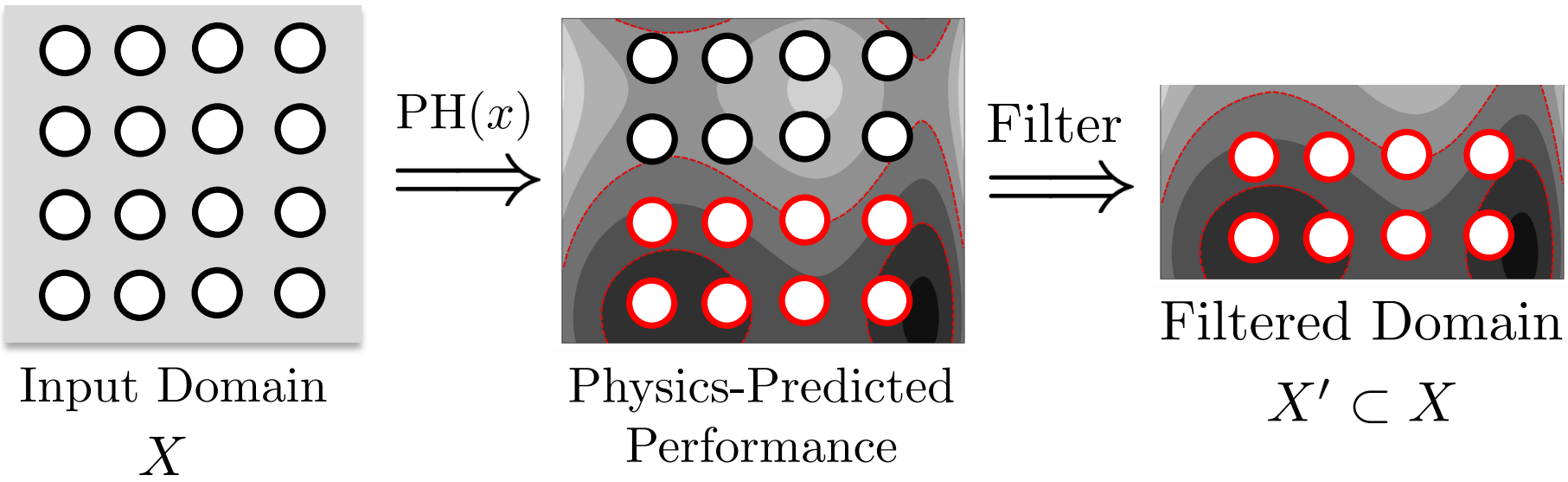
Figure: True (solid) and model (dashed) temperature. Black: training experiment ($T_c = 300$ K) with data; colors: five new coolant temperatures.

Physics-Informed Filtering

- Physics (even approximate) predicts which regions of input domain X are relevant (e.g., promising or safe).
- Physics *narrows down* the training space: it prioritizes or filters points of X to obtain $X' \subset X$; data are collected and models are trained on X'.
- **Learning:** ML model and training do not change; physics acts on data *before* training.
- **Requirements:** physics must be cheap to evaluate over candidate inputs.
- Examples: physics-based partitioning of design spaces in Bayesian optimization, COSMO-RS screening of solvent mixtures (Module 4).

González, L. D., and Zavala, V. M. (2023). New paradigms for exploiting parallel experiments in Bayesian optimization. Comput. Chem. Eng., 170, 108110. doi:10.1016/j.compchemeng.2022.108110

Physics-Informed Filtering



González, L. D., and Zavala, V. M. (2023). New paradigms for exploiting parallel experiments in Bayesian optimization. Comput. Chem. Eng., 170, 108110. doi:10.1016/j.compchemeng.2022.108110

Example: Filtering for CSTR PCML_Module2_UDE_CSTR

- Experiments possible for $T_c \in [280, 310]$ K, but reactor must stay below 335 K (true reactor ignites above 304 K).
- Approximate model keeps T_c with predicted $T \leq 325$ K (10 K margin): $X' = [280, 302.5]$ K.
- Over 20 random designs of 10 experiments: 2.7 unsafe experiments in X vs. 0 in X'; RMSE of GP in safe region 4.9 K vs. 0.8 K.

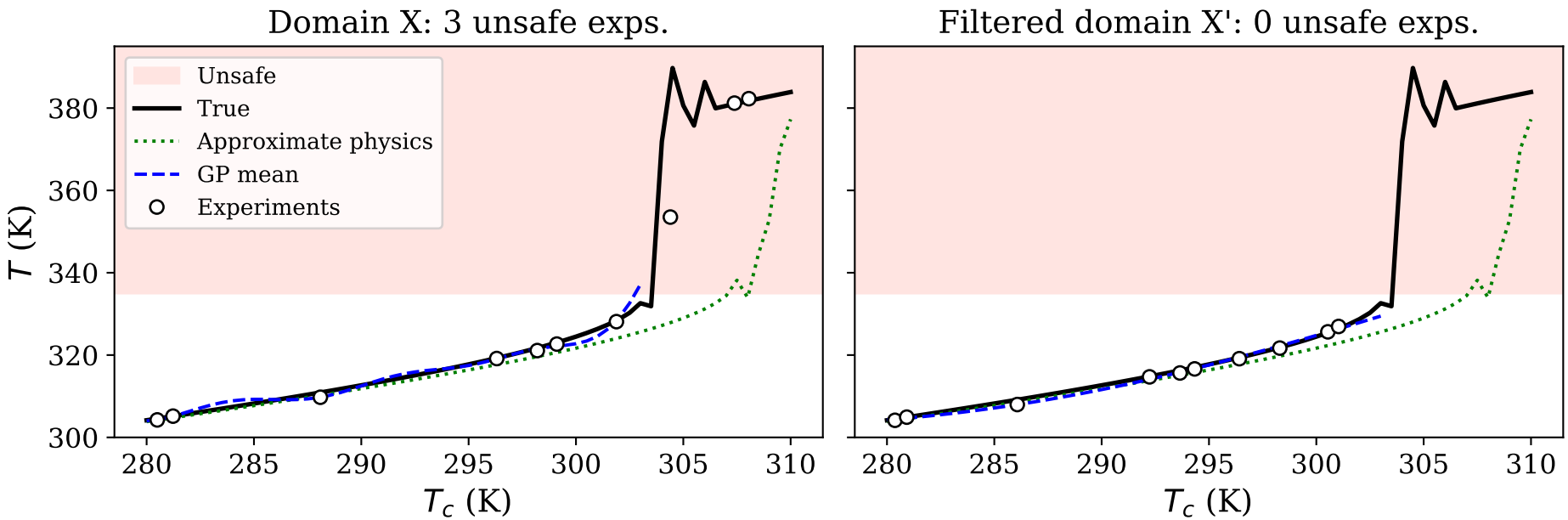


Figure: Random experiments in X (left) and in filtered domain X' (right) with GP fit.

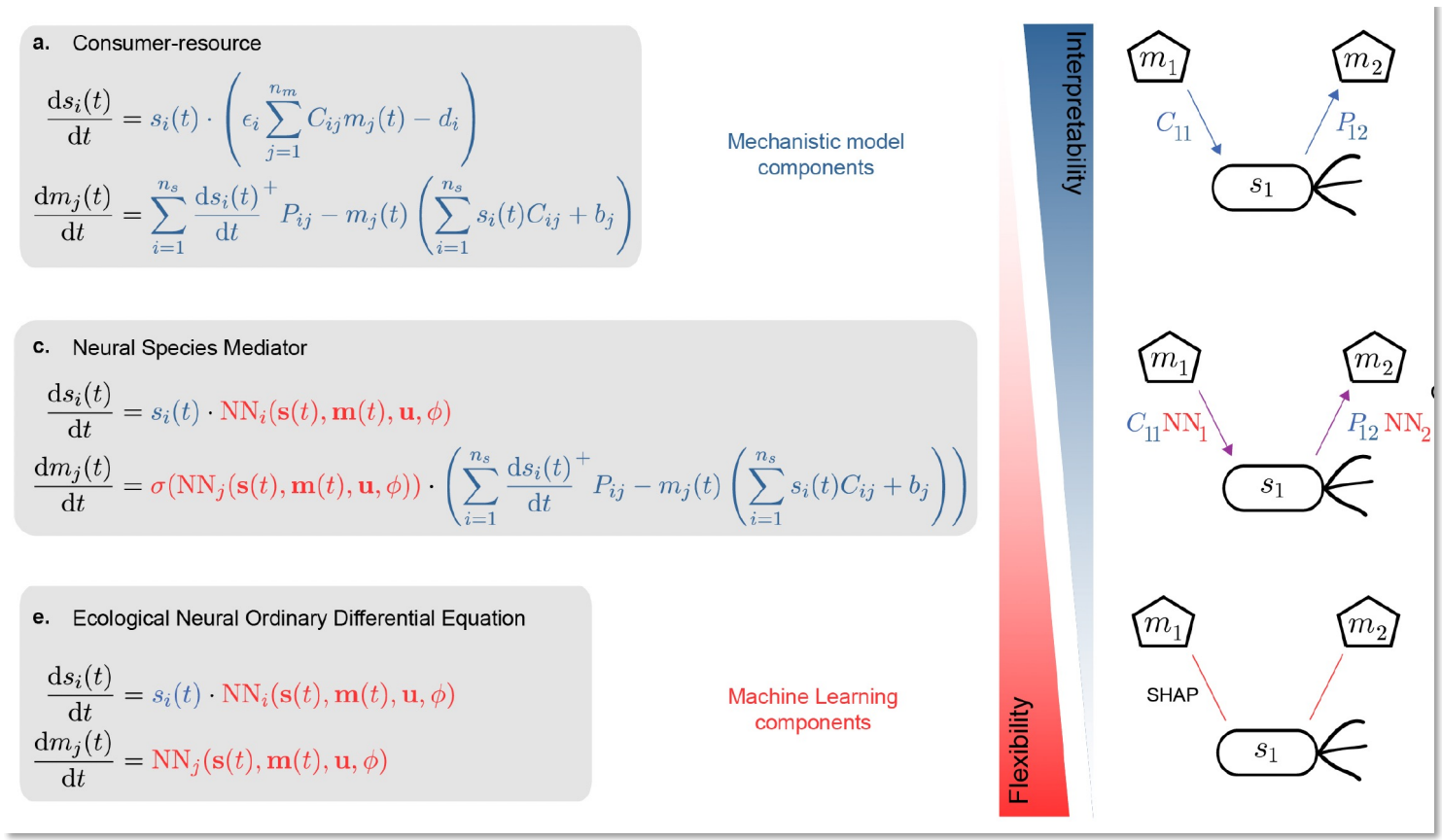
How Architecture Influences Learning

Paradigm	**ML learns**	**Physics needs**	**Physics satisfied**
Additive	Error of physics	Evaluations at data	No (fallback)
PINN	Full output	Differentiable	Approximately
PCNN (explicit)	Part of output	Differentiable	Exactly
PCNN (implicit)	Full output	Projection or Ipopt	Exactly
UDE	Unknown terms	Differentiable ODEs	Exactly (structure)
Filtering	Full output	Cheap evaluations	Not in model

- Physics mitigates limited data, regularizes ML, reduces NN complexity (UDE: 1 experiment, 7 params), and narrows the training space (filtering).
- Physics penalties (PINN) can improve accuracy with few data; hard constraints (PCNN) regularize and guarantee consistency with the imposed physics, not prediction accuracy.
- Paradigms can be combined (e.g., additive PCNN, or filtering + additive GP).

How Much Physics? Microbial Communities

- Goal: predict species abundances $s_i(t)$ and metabolites $m_j(t)$ in microbial communities.
- Range of models from physics to ML:
  - Consumer-resource (CR): mechanistic; interpretable, but rigid.
  - Neural species mediator (NSM): CR structure; NNs capture growth and metabolite production.
  - Ecological neural ODE (eNODE): NNs model all rates; flexible, but harder to interpret.



Thompson, J., Connors, B. M., Zavala, V. M., and Venturelli, O. S. (2026). Physics-constrained neural ordinary differential equation models to discover and predict microbial community dynamics. PNAS, 123(13), e2517661123. doi:10.1073/pnas.2517661123

How Much Physics? Microbial Communities

- Physics alone cannot capture dynamics ($r = 0.36$-0.66); pure ML (eNODE) fits some variables well but others less so (e.g., $r = 0.76$ for FP); hybrid models give the most consistent predictions ($r = 0.82$-0.94).
- UQ via variational inference (Module 3) provides insight into model generalization.
- **Key:** it is *not obvious* how to best fuse physics and ML; which physics to embed and how much fidelity is sufficient remain open questions.

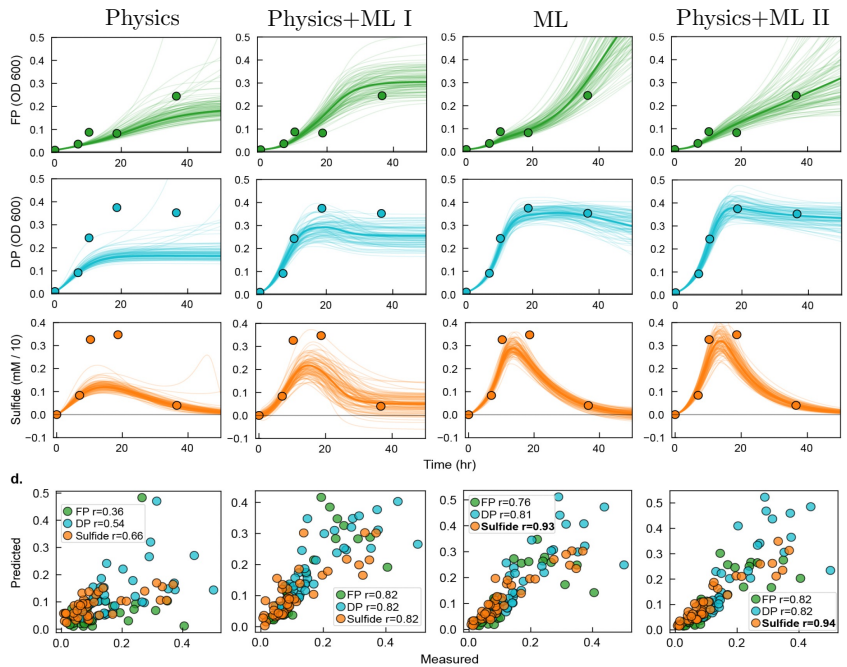


Figure: Predictions (with uncertainty) of physics, hybrid, and ML models for a microbial community.

Key Takeaways

- Physics enters through data (features), structure (additive, composite), loss (informed), constraints (constrained), and training domain (filtering).
- ML models are prone to overfitting and poor generalization; physics *regularizes* them and *narrows down* the space in which we train.
- Each architecture changes the learning problem: what ML learns, what physics is needed, and how training is done.
- Membrane: balances cut purity errors five-fold (10 experiments); CSTR: a 7-param UDE learned dynamics from one experiment (neural ODE could not); filtering avoided unsafe runs.
- How much physics to embed is not obvious; this motivates tools to quantify uncertainty and compare models (Module 3).

References

- Mukherjee, A., and Zavala, V. M. (2026). Physics-constrained machine learning for chemical engineering. Curr. Opin. Chem. Eng., 51, 101228. doi:10.1016/j.coche.2026.101228
- Karniadakis, G. E., et al. (2021). Physics-informed machine learning. Nat. Rev. Phys., 3(6), 422-440. doi:10.1038/s42254-021-00314-5
- Umaña, J. E., Cashen, R. K., Zavala, V. M., and Gebbie, M. A. (2025). Uncovering ion transport mechanisms in ionic liquids using data science. Digital Discovery, 4(6), 1423-1436. doi:10.1039/d4dd00378k
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. (2019). Physics-informed neural networks. J. Comput. Phys., 378, 686-707. doi:10.1016/j.jcp.2018.10.045
- Mukherjee, A., and Bhattacharyya, D. (2024). On the development of steady-state and dynamic mass-constrained neural networks using noisy transient data. Comput. Chem. Eng., 187, 108722. doi:10.1016/j.compchemeng.2024.108722
- Rackauckas, C., et al. (2020). Universal differential equations for scientific machine learning. arXiv:2001.04385. doi:10.48550/arXiv.2001.04385
- Thompson, J., Connors, B. M., Zavala, V. M., and Venturelli, O. S. (2026). Physics-constrained neural ordinary differential equation models to discover and predict microbial community dynamics. PNAS, 123(13), e2517661123. doi:10.1073/pnas.2517661123